Pi Python Workshop

Beginners Track

NLB LearnX

⚠️ **PLEASE DO NOT WRITE ANYTHNG ON THIS HANDOUT**

This structural guide is a reusable classroom resource. Please complete all coding exercises inside your digital workspace.

Phase 1: Foundations & User Interaction

1.1 Variables, Naming Rules, & Number Operations

Think of a variable as a labeled storage drawer in your computer's memory. When you name a drawer and place a piece of information inside it, Python remembers that relationship instantly. For professional clarity, keep names completely lowercase and separate words with underscores (like `user_age`). Standard symbols operate mathematics: add (`+`), subtract (`-`), multiply (`*`), and divide (`/`).

CODE EXAMPLE

```
# Creating variables with numbers
ticket_price = 12
quantity = 4

# Multiply them together and store the result
total_cost = ticket_price * quantity
print(total_cost)
```

Classroom Exercise 1

Create a variable named `hourly_wage` and set it to `25`. Create another variable named `hours_worked` and set it to `40`. Calculate the total weekly pay by multiplying them together, store the final value in a variable called `total_pay`, and print it out to the screen.

1.2 Print Strings & Concatenation

A string is simply a line of raw text wrapped inside quotation marks. If you want to glue strings together back-to-back, you can use the `+` symbol (known as concatenation). To embed active variables seamlessly inside text sentences, use an **f-string**. By adding an `f` directly before your opening quote, you can drop any variable directly into curly braces `{}`.

CODE EXAMPLE

```
first_name = "Alex"

# Using standard text gluing (+)
print("Hello " + first_name + "!")

# Using a modern f-string (much cleaner)
print(f"Welcome back to the terminal, {first_name}.")
```

Classroom Exercise 2

Create a variable named `user_name` and write your own name inside it as text. Use an f-string to build and print a line that reads exactly: `"Hello [user_name], welcome to Python!"`

1.3 Live User Inputs & Type Casting

The `input()` statement pauses your script completely and waits for the user to type text on the keyboard and press Enter. **Crucial Warning:** Python saves everything typed into an input box as a text string, even if they typed numbers! If you intend to run math formulas on a user's answer, you must force convert (or "cast") it into a whole number using `int()`.

> **Teacher's Tip**
>
> Forgetting to convert input text into numbers is the number one reason beginner code crashes. Always wrap your numerical inputs inside `int(input(...))` if you want to calculate balances later.

CODE EXAMPLE

```
# Getting raw text string
favorite_food = input("What do you want to eat? ")

# Getting a number requires wrapping it inside int()
items_requested = int(input("How many servings? "))
```

> **Classroom Exercise 3**
>
> Write a program that prompts a user for a number using `input()`, converts it to an integer, and multiplies it by 10. Print the final calculated value back to the user.

Phase 2: Logical Choices & Data Groups

2.1 Conditional Decisions (If-Else)

An `if-else` block directs your code down alternate execution paths depending on conditions. Python measures blocks using strict indentation (4 blank spaces or 1 tab). Lines indented directly beneath a decision header run only when the parameter condition is met. If that check fails, the program automatically steps down to the `else` structure instead.

CODE EXAMPLE

```
current_score = 72

if current_score >= 50:
    print("Verification passed.")
else:
    print("Action required: Baseline threshold unmet.")
```

Classroom Exercise 4

Set a variable named `temperature` to 32. Design an if-else structural statement: if the temperature value is higher than 30, print `"It's a hot day!"`. Otherwise, print `"The weather is nice."`

2.2 Lists vs. Tuples

When you have a group of multiple items, individual variables become messy. We collect them inside structured containers instead:

- **Lists `[]` (Mutable):** Adaptable sequences. You can freely change elements, swap positions, or add entries while the application runs.
- **Tuples `()` (Immutable):** Frozen arrays. Once created, they are permanently locked in memory. They run incredibly fast and guarantee that vital configuration constants can never be accidentally modified.

CODE EXAMPLE

```
# Modifying an index element inside a mutable list
grocery_items = ["Milk", "Eggs", "Bread"]
grocery_items[1] = "Cheese"  # Swaps out Eggs for Cheese

# Creating an unchangeable structural tuple
fixed_screen_resolution = (1920, 1080)
```

Classroom Exercise 5

Construct a list of three distinct items named `fruit_list`. Overwrite the second fruit inside your list with the string text `"Apple"`. Right below it, save a tuple containing two fixed numbers named `spawn_point = (10, 20)`.

Phase 3: Automation & Loop Machinery

3.1 Definite Loops (For) & Indefinite Loops (While)

Loops eliminate tedious copy-and-paste routines by processing tasks repeatedly. A `for` loop iterates through a fixed list sequence, running a specific number of times until the container is empty. A `while` loop runs endlessly as long as its check statement evaluates to True. You must alter a variable inside the while loop to make the statement false eventually, otherwise your application will get stuck in an infinite loop.

CODE EXAMPLE

```
# The For loop steps through a sequence completely
for color in ["Red", "Green", "Blue"]:
    print(f"Color found: {color}")

# The While loop keeps checking a live state counter
timer = 3
while timer > 0:
    print(timer)
    timer = timer - 1
```

Classroom Exercise 6

Write a `for` loop that cycles through your previously generated `fruit_list` and prints every fruit element individually on a clean line.

Classroom Exercise 7

Establish a loop counter variable named `countdown = 5`. Build a `while` loop that prints out the current countdown status value, reduces it by 1 on every pass, and exits cleanly. Once fully done, print `"Blast off!"`.

Phase 4: Reusable Recipes & System Libraries

4.1 Defining Custom Functions

A function is a named recipe block of code. Creating a function via the `def` keyword commands Python to memorize an operational sequence without executing it immediately. Functions take in values through **parameters** and send calculated results outward using the **`return`** keyword.

CODE EXAMPLE

```
def multiply_by_two(number):
    result = number * 2
    return result

# Calling our function logic out later
saved_output = multiply_by_two(15)
print(saved_output)  # Prints 30
```

Classroom Exercise 8

Write a custom function named `calculate_area` that accepts two incoming values: `length` and `width`. Have the function compute their area product and `return` that value. Call your custom function using the inputs 5 and 10, then print out the result.

4.2 Importing Modular Engines (Random & Time)

Python leaves advanced software toolsets packed away inside its standard library to optimize runtime speeds. To utilize these utilities, pull them into your workspace using the `import` statement. We use `random.randint(a, b)` to instantly generate unpredictable whole numbers, and `time.sleep(seconds)` to pause thread processing execution deliberately.

CODE EXAMPLE

```
import random
import time

print("Connecting...")
time.sleep(2.0)  # Freezes terminal activity for exactly 2 seconds
random_die_roll = random.randint(1, 6)
print(f"Result: {random_die_roll}")
```

Classroom Exercise 9

Import the `random` and `time` engine libraries. Write a short script that displays `"Rolling dice..."`, pauses execution processing for exactly 2 seconds, and then outputs a random selection integer scale ranging between 1 and 6.

🏆 Capstone Project Challenge: The Number Guessing Game

This final graduation script seamlessly unifies every single foundational skill you have mastered today into a fully operational text application. Review how inputs, numbers, string tracking, logic gates, conditional while loops, and library imports coordinate together in unity.

PRODUCTION BLUEPRINT

```
import random
import time

# Setup state variables and generate the secret value
hidden_number = random.randint(1, 10)
guess_attempts = 0

print("=== CORE PYTHON TRAINING CAPSTONE INITIALIZED ===")
user_name = input("Please enter your name: ")
print(f"Welcome, {user_name}! I have locked a hidden integer between 1 and
10.")
time.sleep(1.0)

# Run an interactive indefinite loop until conditions match
while guess_attempts < 3:
    # Accept user guess input and instantly cast it into an integer
    current_guess = int(input("Take an active guess (1-10): "))
    guess_attempts = guess_attempts + 1

    # Conditional branching checks
    if current_guess == hidden_number:
        print(f"🎉 Fantastic job, {user_name}! You decoded it in
{guess_attempts} attempts!")
        break
    elif current_guess < hidden_number:
        print("Too low! Try adjusting higher.")
    else:
        print("Too high! Try adjusting lower.")

    time.sleep(0.5)

# Resolution screen state logic check
if current_guess != hidden_number:
    print(f"
Game Over! The target number was securely locked at: {hidden_number}.")
print("Thank you for learning Python basics with us!")
```